

# AI Gateway with Intelligent Routing, Safe Caching, and Scalable Model Serving

Aneesh Kadiyala (EE23BTECH11007)  
Naga Rohith Kumar Jakkala (CS23BTECH11021)

September 18, 2026

## 1 Background

Choosing LLMs today involves considering several factors: the model's capabilities and how well it fits a specific task, cost per token, cost per task (models with cheaper tokens may generate more tokens for the same task, increasing overall cost), availability, throughput (the rate at which tokens are generated for a given model and provider), and latency (the time it takes for the first token and the full response to be generated). To add to the complexity, various providers often offer different API endpoints and different ways to cache tokens or configure the model's behaviour.

## 2 Problem Statement

To simplify these decisions and prevent being locked into a single provider or creating an extremely large codebase for each project to support multiple providers, an AI gateway that offers a single API and routes requests to the appropriate provider behind the scenes is the need of the hour. Additionally, an intelligent router that can automatically select a model and provider, taking the above parameters into account, would reduce the cognitive load of switching models for different tasks and provide a set-and-forget experience.

### 2.1 Scope

#### **AI Gateway**

A stable and resilient AI gateway that offers a single API following existing LLM API conventions and routes requests to one of multiple providers (at least one external and one self-hosted), enforces appropriate rate limits and quotas, and performs bounded retry or routes to a fallback in case of provider-side rate limiting or provider outage.

#### **Intelligent Routing**

An automatic router that picks a model and provider based on task, quality, latency, privacy/safety, cost or availability signals.

#### **Caching**

Offer users the ability to enable KV caching with explicit cache keys and invalidation rules.

### **Observability**

Offer users the ability to transparently view routing decisions, latency, cache outcomes, token usage, failure modes and correlation traces without exposing secrets by default.

### **SLO-Aware Serving and Load-Balancing**

Maintain replica queue state and request size and GPU/memory pressure-informed scheduling.

### **Benchmarking**

Compare the routing policy against static routing and the load-balancing policy against round-robin and least-loaded baselines.

## **2.2 Key Challenges**

### **Low-Latency Routing Decisions**

Evaluating routing metrics (task classification, cost, availability, latency predictions) dynamically without introducing unacceptable overhead to time-to-first-token.

### **Safe and Deterministic Caching**

Managing cache consistency, semantic vs. exact matching, handling cache invalidation rules safely, and preventing cross-tenant data leakage or serving stale context.

### **Provider Heterogeneity and Normalization**

Abstracting divergent provider APIs, error schemas, tokenization nuances, and streaming response formats into a unified standard.

### **Dynamic Replica State Tracking**

Reliably tracking queue depth, request payload size, and GPU/VRAM memory pressure across self-hosted replicas in real time without high polling overhead.

### **Fault Handling and Graceful Degradation**

Designing transparent fallbacks and bounded retries that handle partial failures, context-window mismatches, and sudden rate limits without compounding downstream latency.

### **Zero-Leak Observability**

Emitting granular traces and telemetry (cache hit/miss, routing decisions, token counts) while strictly masking sensitive user inputs, PII, and API keys.

## **3 Expected Output**

### **3.1 Software Artifacts**

- A deployable AI Gateway microservice (containerized via Docker) exposing an OpenAI-compatible `/v1/chat/completions` endpoint.
- A unified configuration file allowing users to register upstream providers, set fallback policies, define rate limits, and specify caching rules.

- A CLI interface allowing users to modify configuration variables, generate keys, enforce limits on keys and view audit logs for observability.

## **3.2 Empirical Benchmarking**

- Comparative latency and cost curves demonstrating routing decisions versus a baseline static configuration across standard workloads (using small subsets of MT-Bench or MMLU).
- Load-test profiles comparing peak throughput and SLO compliance (P95/P99 latency) of the queue-aware scheduler against round-robin and least-loaded algorithms under synthetic traffic bursts.

## **3.3 Verification and Audit Artifacts**

- An end-to-end trace log sample showing request routing decisions, sanitization (redacted secrets/PII), and cache hit/miss statuses.
- A documented test suite validating failover behavior under simulated provider outages and rate-limit errors (HTTP 429).